

Übersetzung von Vorlesungsunterlagen in TypeScript TheoInf I + II

Projektarbeit

des Studiengangs Angewandte Informatik
an der Dualen Hochschule Baden-Württemberg Mannheim

von
Tobias Neithöfer und Leo Vatter

Abgabedatum: 15. April 2026

Bearbeitungszeitraum

01.10.2025 - 15.04.2026

Kurs

TINF23AI2

Gutachter

Prof. Dr. Karl Stroetmann

Unterschrift Gutachter

Erklärung

gemäß § 35 (1) der „Studien- und Prüfungsordnung DHBW Technik“ vom 7. März 2024.

Wir haben die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.

Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.*

* falls beide Fassungen gefordert sind

Mannheim, 5. Februar 2026

Ort, Datum

Unterschrift

Abstrakt

Kurze Zusammenfassung der Arbeit.

Abstract

Short summary of the paper.

Inhaltsverzeichnis

Abkürzungsverzeichnis	VI
-----------------------	----

Abbildungsverzeichnis	VIII
-----------------------	------

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
2	Grundlagen	4
2.1	Anaconda	4
2.2	Interaktive Programmierung mit Jupyter	5
2.3	TypeScript im Data-Science-Kontext	6
2.3.1	Herausforderungen und Bibliotheks-Ökosystem	6
2.3.2	Syntaxvergleich und Typisierungskonzepte	7
2.4	Setup und Toolchain	8
2.4.1	Architektur der Laufzeitumgebung	8
2.4.2	Implementierung der Umgebung	9
2.4.3	Validierung der Toolchain	10
3	Einführung in TypeScript	11
3.1	Primitive Datentypen und Arithmetik	12
3.2	Ausgabe und Zeichenkettenliterals	13
3.3	Arithmetische Operationen und Variablenzuweisung	14
3.4	Arrays	15
3.5	Tupel	20
3.6	Objekte	21
3.7	Boolesche Werte und Operatoren	23
3.8	Kontrollstrukturen	24
3.9	Numerische Funktionen	26
4	Methodische Vorgehensweise	28
4.1	Github Projects	28
4.2	Auswahl der Lehrmaterialien	28

4.3	Der Übersetzungs-Workflow	28
4.4	Einsatz von KI-gestützten Tools	28
5	Case Studies	29
5.1	CNF	29
5.2	Graphviz	29
5.3	Wiederverwendung von Notebooks	29
5.4	Recursive Set	29
6	Evaluierung und Ergebnisse	30
6.1	Laufzeit	30
6.2	Lesbarkeit	30
6.3	Effizienz	30
7	Fazit und Ausblick	31
	Literaturverzeichnis	IX

Abkürzungsverzeichnis

DHBW Duale Hochschule Baden-Württemberg

KI Künstliche Intelligenz

JSON JavaScript Object Notation

List of Listings

1	Python Tupel	7
2	TypeScript Tupel Implementierung	8
3	Dictionaries in TypeScript	8
4	Bereitstellung der Laufzeitumgebung	9
5	Integration des TypeScript-Kernels	9
6	Installation der Projekt-Abhängigkeiten	10
7	Typüberprüfung numerischer Literale	12
8	Demonstration des Präzisionsverlusts bei Number	12
9	Verwendung von BigInt	12
10	Ausgabe und Definition von Strings	13
11	Verwendung von Template Strings	14
12	Ganzzahldivision und Modulo	14
13	Konstanten vs. Variablen	15
14	Definition eines leeren Arrays	15
15	Array mit numerischen Elementen	15
16	Zugriff auf Array-Elemente	16
17	Modifikation eines Array-Elements	16
18	Verkettung mit concat	16
19	Verkettung mit dem Spread-Operator	16
20	Bestimmung der Array-Länge	16
21	Signatur <i>filter</i>	17
22	Filtern gerader Zahlen	17
23	Signatur from	18
24	Erzeugung einer Zahlenfolge	18
25	Quadrieren von Zahlen mit map	19
26	Signatur from	19
27	Summierung mit reduce	19
28	Einfaches Array Destructuring	20
29	Destructuring mit Rest-Operator	20
30	Definition und Initialisierung eines Tupels	21
31	Zugriff auf Tupel-Elemente	21

32	Objektdefinition	21
33	Zugriffsarten auf Objekteigenschaften	22
34	Modifikation von Eigenschaften	22
35	Verschachtelte Objekte	22
36	Zugriff auf verschachtelte Daten	22
37	Logische Verknüpfungen	23
38	If-Else Verzweigung	24
39	Switch-Statement als Alternative	25
40	Optimiertes Sieb des Eratosthenes	25
1	TypeScript: Beispiel mit Interface und Funktion	31

1 Einleitung

Die vorliegende Studienarbeit dokumentiert die systematische Portierung von Jupyter-Notebooks aus der Programmiersprache Python nach TypeScript. Diese Notebooks bilden die Grundlage für die Lehrveranstaltungen „Theoretische Informatik I“ (Logik) und „Theoretische Informatik II“ (Algorithmen und Datenstrukturen) und dienen als interaktive Vorlesungsunterlagen zur Vermittlung theoretischer Konzepte der Informatik.

Die Verwendung von Jupyter-Notebooks in dem Informatikstudium hat sich in den letzten Jahren als effektive Methode etabliert, um theoretische Konzepte mit praktischen Implementierungen zu verbinden [3]. Während Python aufgrund seiner Verbreitung und einfachen Syntax häufig als erste Programmiersprache gewählt wird, stellt die dynamische Typisierung in komplexeren Projekten zunehmend eine Herausforderung dar. Diese Problematik führt zur zentralen Fragestellung dieser Arbeit: Lässt sich durch den Einsatz statischer Typisierung die Codequalität, Nachvollziehbarkeit und Effizienz in Lehrveranstaltungen verbessern, ohne dabei die didaktische Zugänglichkeit zu beeinträchtigen?

1.1 Problemstellung

Die aktuelle Implementierung der Vorlesungsunterlagen basiert auf 80 Jupyter-Notebooks. Prof. Dr. Karl Stroetmann formulierte die Aufgabenstellung wie folgt: „Meine Vorlesungen Theoretische Informatik I+II [2] [1] verwenden eine Reihe von Jupyter-Notebooks, die auf der Sprache Python basieren. Die Sprache Python ist nur dynamisch getypt. Zwar gibt es für Python den Typ-Checker MyPy, aber das von MyPy unterstützte Typsystem ist wesentlich schwerfälliger als das Typsystem von TypeScript. Mein Ziel ist es daher, meine Vorlesungen von Python auf TypeScript umzustellen.“

Die zentrale Problematik ergibt sich aus mehreren Aspekten:

Mangelnde Typsicherheit in der informatischen Ausbildung

Python ist eine dynamisch typisierte Sprache, bei der Typfehler erst zur Laufzeit erkannt werden. Dies führt insbesondere in der Lehre zu Herausforderungen, da Studierende häufig erst spät, nach Ausführung des Codes, auf Typinkompatibilitäten stoßen. Während Tools wie MyPy eine nachträgliche statische Typprüfung ermöglichen, bleibt die Integration in bestehenden Python-Projekte oft komplex und

das Typsystem weniger ausdrucksstark als bei nativ statisch typisierten Sprachen [6]. Diese verzögerte Fehlerkennung kann das Verständnis für Typsysteme und deren Bedeutung in der Softwareentwicklung beeinträchtigen.

Eingeschränkte Möglichkeiten der statischen Codeanalyse

TypeScript bietet als Superset von JavaScript ein robustes, strukturelles Typsystem, das bereits während der Entwicklung umfassende Fehlerprüfungen ermöglicht [6]. Die statische Typprüfung zur Compile-Zeit erlaubt es, viele potenzielle Laufzeitfehler bereits im Vorfeld zu identifizieren und trägt somit zu einer verbesserten Code-Qualität bei. In der Lehre kann dies Studierenden ein tieferes Verständnis für Datenstrukturen und Algorithmen vermitteln, da Typen explizit deklariert und durch den Compiler überprüft werden.

Umfang des Portierungsprojekts

Mit insgesamt 80 Notebooks stellt der Umfang der zu übersetzenden Materialien eine erhebliche Herausforderung dar. Diese Menge erfordert eine systematische Vorgehensweise sowie den strategischen Einsatz von KI-gestützten Übersetzungstools, um Konsistenz und Qualität über alle Notebooks hinweg zu gewährleisten.

1.2 Zielsetzung

Die vorliegende Arbeit verfolgt mehrere miteinander verbundene Zielsetzungen. Das vom Dozenten vorgegebene Hauptziel besteht in der vollständigen Übersetzung der 80 Python-basierten Jupyter-Notebooks nach TypeScript unter Verwendung von Künstliche Intelligenz (KI)-Tools wie ChatGPT, Gemini, DeepSeek oder Grok. Diese Portierung soll die Vorlesungsmaterialien auf eine Sprache mit statischer Typisierung überführen und damit die didaktischen Möglichkeiten erweitern.

Ein zentrales Forschungsziel liegt in der Untersuchung, ob die in TypeScript implementierten Programme eine vergleichbare oder möglicherweise verbesserte Verständlichkeit aufweisen als die ursprünglichen Python-Implementierungen. Diese Fragestellung ist von besonderer Relevanz für die Lehrpraxis, da die Zugänglichkeit des Codes für Studierende nicht durch die erhöhte Typsicherheit beeinträchtigt werden sollte.

Zusätzlich zur qualitativen Bewertung soll anhand der übersetzten Notebooks ein quantitativer Performanzvergleich zwischen Python und TypeScript durchgeführt werden. Dieser Vergleich kann Aufschluss über die praktische Eignung beider Sprachen für rechenintensive Algorithmen und Datenstrukturen geben. Die Arbeit folgt dabei einem systematischen Ansatz, der den Einsatz von KI-gestützten Übersetzungstools mit manueller Überprüfung und Optimierung kombiniert, wobei Best Practices für die TypeScript-Entwicklung berücksichtigt und die Übersetzungsqualität kontinuierlich überprüft werden.

2 Grundlagen

Dieses Kapitel erläutert die technologischen Grundlagen und Werkzeuge, die für die Durchführung dieser Studienarbeit relevant sind. Zunächst wird die Distributionsplattform Anaconda vorgestellt, gefolgt von einer detaillierten Betrachtung der interaktiven Entwicklungsumgebung Jupyter. Anschließend erfolgt eine Einführung in die Zielsprache TypeScript sowie der Vergleich mit Python. Den Abschluss bildet die Beschreibung der spezifischen Setup- und Toolchain-Konfiguration.

2.1 Anaconda

Anaconda ist eine weit verbreitete Open-Source-Distribution für die Programmiersprachen Python und R, die speziell für Aufgaben in den Bereichen Data Science, maschinelles Lernen und wissenschaftliches Rechnen entwickelt wurde. Sie vereinfacht das Paketmanagement und die Bereitstellung von Softwareumgebungen erheblich, was insbesondere in komplexen wissenschaftlichen Projekten von Vorteil ist. Kernkomponente der Distribution ist der Paketmanager `conda`. Dieser ermöglicht es, Softwarepakete und deren Abhängigkeiten plattformübergreifend zu installieren, zu aktualisieren und zu verwalten. Ein wesentliches Merkmal von Anaconda ist die Fähigkeit, isolierte Umgebungen (Environments) zu erstellen. Diese Isolation verhindert Konflikte zwischen verschiedenen Versionen von Bibliotheken, die für unterschiedliche Projekte benötigt werden könnten.

Für die vorliegende Arbeit dient Anaconda als primäre Basisinfrastruktur. Obwohl das Ziel der Portierung auf TypeScript liegt, wird Anaconda ebenfalls verwendet, um die ursprünglichen Python-Notebooks auszuführen und zu validieren. Darüber hinaus fungiert das Anaconda Prompt (bzw. das Terminal) als Schnittstelle für systemweite Befehle. Hierüber erfolgen auch die notwendigen Installationen für das TypeScript-Ökosystem, wie beispielsweise die Ausführung von `npm install`, um Node.js-Pakete global oder lokal verfügbar zu machen. Die Verwaltung der Node.js-Laufzeitumgebung kann ebenfalls in die bestehende Infrastruktur integriert werden, was Anaconda zu einem vielseitigen Werkzeug macht.

2.2 Interaktive Programmierung mit Jupyter

Das Jupyter Notebook ist eine webbasierte, interaktive Entwicklungsumgebung, die es ermöglicht, Dokumente zu erstellen, die sowohl ausführbaren Code als auch formatierten Text, mathematische Formeln und Visualisierungen enthalten. Der Name „Jupyter“ ist ein Akronym, das sich ursprünglich aus den Kernsprachen Julia, Python und R ableitete, mittlerweile aber eine Vielzahl weiterer Programmiersprachen unterstützt.

Die Architektur von Jupyter basiert auf einem Client-Server-Modell, das die Trennung zwischen der Benutzeroberfläche und der Code-Ausführung ermöglicht.

- Das Frontend (Client): Der Benutzer interagiert über einen Webbrowser mit dem Notebook. Die Oberfläche erlaubt die Eingabe von Code in Zellen sowie die Gestaltung von Textbereichen mittels Markdown.
- Der Notebook-Server: Dieser Server verwaltet die Dateien und vermittelt die Kommunikation zwischen dem Frontend und dem ausführenden Kernel.
- Das Dateiformat (.ipynb): Jupyter Notebooks werden als JavaScript Object Notation (JSON)-Dateien mit der Endung `.ipynb` gespeichert. Diese strukturierte Textdatei enthält den gesamten Inhalt des Notebooks, Code, Markdown-Text, Metadaten sowie die Ausgabeartefakte (z. B. Konsolenausgaben oder Grafiken), in einem standardisierten Format. Dies erleichtert die Versionierung und den Austausch von wissenschaftlichen Ergebnissen.

Ein zentrales Element der Jupyter-Architektur ist der Kernel. Der Kernel ist ein separater Prozess, der für die Ausführung des Codes zuständig ist, den der Benutzer im Notebook eingibt. Wenn eine Code-Zelle ausgeführt wird, sendet das Frontend den Code an den Kernel. Dieser führt die Anweisungen aus und sendet das Ergebnis zurück an das Frontend zur Darstellung. Für diese Studienarbeit ist das Kernel-Konzept von entscheidender Bedeutung:

Während die ursprünglichen Vorlesungsunterlagen auf dem Standard-Python-Kernel (ipykernel) basieren, erfordert die Portierung auf TypeScript die Integration eines dedizierten TypeScript-Kernels (tslab). Diese Architektur ermöglicht den Wechsel der Programmiersprache innerhalb derselben Benutzeroberfläche durch bloßen Austausch des Ba-

ckends. Die technische Integration des TypeScript-Kernels wird in Abschnitt 2.5 [Setup und Toolchain](#) detailliert beschrieben.

2.3 TypeScript im Data-Science-Kontext

Während Python als De-facto-Standard im Bereich Data Science gilt, gewinnt das TypeScript-Ökosystem zunehmend an Bedeutung, insbesondere wenn Modelle direkt in Webanwendungen integriert oder visualisiert werden sollen.

2.3.1 Herausforderungen und Bibliotheks-Ökosystem

Eine wesentliche Herausforderung bei der Migration von Python zu TypeScript ist die Diskrepanz in der Standardbibliothek. Während Python mit Modulen wie `heapq` oder einer nativen Unterstützung für komplexe Mengenoperationen und Tupel ausgestattet ist, fehlen diese Konstrukte in der Standard-Laufzeitumgebung von TypeScript oft oder sind für wissenschaftliche Zwecke nicht performant genug implementiert. Zudem existiert zwar ein reiches Ökosystem (npm), dieses ist jedoch stark fragmentiert, im Gegensatz zu den monolithischen Standard-Paketen in Python wie beispielsweise NumPy.

Um die Funktionalität der Python-Vorlesungsunterlagen zu replizieren, wird in dieser Arbeit auf eine Auswahl spezialisierter Bibliotheken zurückgegriffen:

Mathematik und Logik:

Für numerische Stabilität und symbolische Berechnungen kommen `mathjs` (umfassende Mathematik-Bibliothek) und `fraction.js` (für rationale Zahlen zur Vermeidung von Fließkommafehlern) zum Einsatz. Für Constraint-Solving-Probleme und logische Beweise werden der `logic-solver` sowie der `z3-solver` (ein Theorem-Prover von Microsoft Research) eingebunden.

Datenstrukturen:

Da TypeScript keine native Priority Queue besitzt (äquivalent zu Pythons `heapq`), wird `heap-js` verwendet. Eine Besonderheit stellt die Bibliothek `recursive-set` dar. Native JavaScript `Sets` prüfen Objekte auf Referenzgleichheit, nicht auf Wertegleichheit. Das bedeutet, zwei inhaltlich identische Objekte werden als unterschiedlich behandelt. `recursive-set` ermöglicht echte Mengenoperationen auf Basis der tiefen Gleichheit (Deep Equality).

Visualisierung und Medien:

Zur Generierung von Graphen wird `@hpc-js/wasm` genutzt. Dies ist eine WebAssembly-Portierung von Graphviz, die das Rendern von DOT-Sprache ermöglicht, ohne dass eine lokale `graphviz.exe`-Installation auf dem Host-System notwendig ist – ein entscheidender Vorteil für die Portabilität des Notebooks. Für die Bildmanipulation wird `pngjs` verwendet.

Type Definitions:

Ein Spezifikum der TypeScript-Entwicklung ist die Notwendigkeit separater Typ-Definitionen für Bibliotheken, die ursprünglich in reinem JavaScript geschrieben wurden. So muss beispielsweise für die Bildverarbeitung zusätzlich das Paket `@types/pngjs` als *Dev-Dependency* installiert werden, um die statische Typprüfung zu ermöglichen.

@TobiKnight

2.3.2 Syntaxvergleich und Typisierungskonzepte

Der fundamentale Unterschied zwischen Python und TypeScript liegt im Zeitpunkt und der Strenge der Typprüfung.

Statisch vs. Dynamisch / MyPy vs. TypeScript Compiler

Python ist dynamisch typisiert; Typen werden zur Laufzeit ausgewertet. Tools wie `mypy` bieten zwar Type-Hinting, diese sind jedoch optional und haben keinen Einfluss auf die Ausführung. TypeScript hingegen ist ein statisch typisierter Superset von JavaScript. Der TypeScript-Compiler (tsc) prüft die Typen zur Kompilierzeit (Transpilierung) und entfernt diese anschließend vollständig (Type Erasure). Das Resultat ist reines JavaScript.

Tupel und Datenstrukturen

Ein markanter Unterschied zeigt sich bei unveränderlichen Listen (Tupeln). In Python sind Tupel native, unveränderliche Datentypen.

Listing 1: Python Tupel

```
# Python: Natives, unveränderliches Tupel  
point = (10, 20)
```

In TypeScript existieren Tupel nur als Typ-Konstrukt zur Kompilierzeit. Zur Laufzeit sind es gewöhnliche JavaScript-Arrays. Um die Unveränderlichkeit (Immutability) zu gewährleisten, muss der Modifikator `readonly` explizit verwendet werden:

```

1 // TypeScript: Array mit fester Länge und Typisierung
2 // 'readonly' verhindert Laufzeitmanipulation (zur Compilezeit)
3 const point: readonly [number, number] = [10, 20];

```

Listing 2: TypeScript Tupel Implementierung

Dictionaries vs. Dynamische Objekte

Was in Python das `dictionary` ist, wird in TypeScript oft durch einfache Objekt-Literale oder die `Record`-Utility-Types abgebildet. Anders als in Python, wo Dictionaries beliebige hashbare Objekte als Keys nutzen können, sind Objektschlüssel in JavaScript traditionell auf Strings und Symbole beschränkt. Für komplexe Keys muss in TypeScript explizit die `Map`-Datenstruktur verwendet werden.

```

1 // Einfaches Key-Value Mapping (String zu Number)
2 const scores: Record<string, number> = {
3     Alice : 100,
4     Bob   : 85
5 };
6
7 // Map für komplexere Anwendungsfälle (erhält Einfü
   gereihenfolge)
8 const complexMap = new Map<object, number>();

```

Listing 3: Dictionaries in TypeScript

2.4 Setup und Toolchain

Die Reproduzierbarkeit der Ergebnisse und die Stabilität der Entwicklungsumgebung sind essenziell für die Validierung der portierten Artefakte. Dieses Unterkapitel beschreibt die Architektur der verwendeten Toolchain und dokumentiert die notwendigen Konfigurationsschritte, wie sie in der [README.md](#) der Arbeit definiert sind.

2.4.1 Architektur der Laufzeitumgebung

Die technische Infrastruktur basiert auf einer hybriden Architektur, die das Python-zentrierte Ökosystem von Anaconda mit der JavaScript-basierten Node.js-Laufzeitumgebung

verbindet. Diese Trennung ist notwendig, da Jupyter ursprünglich für Python konzipiert wurde, TypeScript jedoch eine Node.js-Laufzeit für die Transpilierung und Ausführung benötigt.

Die Toolchain gliedert sich in drei Schichten:

1. **Basisinfrastruktur (Anaconda):** Bereitstellung von Python, Jupyter (`nbclassic`) und Verwaltung der Systempfade.
2. **Laufzeitumgebung (Node.js):** Ausführungsumgebung für den TypeScript-Compiler und die genutzten Bibliotheken.
3. **Kernel-Layer (tslab):** Eine Middleware, die das Jupyter-Messaging-Protokoll implementiert und TypeScript-Codezellen an die Node.js-Laufzeit weiterleitet.

2.4.2 Implementierung der Umgebung

Die Einrichtung der Umgebung erfolgt deklarativ über die Kommandozeile der Anaconda-Distribution. Zunächst wird die Laufzeitumgebung durch die Installation von Node.js und dem klassischen Jupyter-Frontend vorbereitet. Dies schafft die Voraussetzungen für die Integration nicht-nativer Kernel.

```
1  conda activate base
2  conda install nodejs
3  conda install -c conda-forge nbclassic
```

Listing 4: Bereitstellung der Laufzeitumgebung

Für die Ausführung von TypeScript innerhalb der Notebooks wird der Kernel `tslab` verwendet. Im Gegensatz zu reinen JavaScript-Kerneln (wie `IJavascript`) ermöglicht `tslab` eine direkte Typ-Prüfung innerhalb der Zellen, was für die didaktische Zielsetzung der Arbeit, die Vermittlung von Typsicherheit, unabdingbar ist.

```
1  tslab install
2  npm install -g tslab
3  npm install typescript
```

Listing 5: Integration des TypeScript-Kernels

Die fachlichen Anforderungen der Vorlesungen erfordern spezifische Bibliotheken, die über den Paketmanager `npm` bezogen werden. Hierbei wird strikt zwischen Laufzeit-

Abhängigkeiten (z. B. `mathjs` für Numerik, `recursive-set` für Mengenlehre) und Entwicklungs-Abhängigkeiten (z. B. `@types/pngjs`) unterschieden. Letztere sind notwendig, um für ursprünglich in JavaScript geschriebene Bibliotheken eine statische Typprüfung zu ermöglichen.

```
1 // Mathematische und logische Kernkomponenten
2 npm install fraction.js mathjs logic-solver z3-solver
3
4 // Datenstrukturen und Mengenoperationen
5 npm install heap-js recursive-set@8.0.0
6
7 // Visualisierung und Medienverarbeitung
8 npm install @hpcc-js/wasm pngjs
9 npm i --save-dev @types/pngjs
```

Listing 6: Installation der Projekt-Abhängigkeiten

2.4.3 Validierung der Toolchain

Um sicherzustellen, dass die Interaktion zwischen dem Jupyter-Frontend, dem `tslab`-Kernel und den installierten Node-Modulen korrekt funktioniert, wurde ein dediziertes Testverfahren etabliert. Das Referenz-Notebook [Test-Libraries.ipynb](#) führt einen systematischen Import-Test aller Komponenten durch. Dieses Notebook verifiziert, dass:

- der Kernel korrekt registriert ist und TypeScript-Syntax verarbeitet,
- alle externen Module im Pfad der Laufzeitumgebung gefunden werden,
- die Typdefinitionen korrekt aufgelöst werden.

Erst nach erfolgreicher Ausführung dieses Notebooks gilt die Umgebung als valide für die Bearbeitung der Vorlesungsunterlagen.

3 Einführung in TypeScript

TypeScript wurde von Microsoft entwickelt und erstmals im Oktober 2012 der Öffentlichkeit vorgestellt. Die Entwicklung stand unter der Leitung von Anders Hejlsberg, einem renommierten Softwarearchitekten, der zuvor bereits maßgeblich an der Gestaltung von C#, Delphi und Turbo Pascal beteiligt war. [4] [5] Der primäre Beweggrund für die Schaffung dieser neuen Sprache lag in den Defiziten von JavaScript bei der Entwicklung großer, komplexer Anwendungen (*Large-Scale Application Development*) [4].

Als Antwort auf diese Herausforderungen ist TypeScript als eine Erweiterung von JavaScript konzipiert, mit dem expliziten Ziel, die Entwicklung großer und langfristig wartbarer Anwendungen zu erleichtern [web:1]. Syntaktisch ist TypeScript als Superset von ECMAScript 5 definiert, sodass jedes gültige JavaScript-Programm zugleich ein gültiges TypeScript-Programm ist [web:1]. Zusätzlich zu JavaScript ergänzt TypeScript unter anderem ein Modulsystem, Klassen, Interfaces sowie ein reichhaltiges graduelles Typsystem, das eine schrittweise (inkrementelle) Typisierung bestehender Codebasen unterstützt. [6] Ein wesentliches Entwurfsprinzip ist dabei die *Typauslöschung* (type erasure): Die Typinformationen werden vom Compiler zur statischen Analyse genutzt, sind jedoch nicht Bestandteil des erzeugten JavaScript-Codes, sodass zur Laufzeit keine TypeScript-Typen repräsentiert oder automatisch geprüft werden [web:1]. Der TypeScript-Compiler überprüft TypeScript-Programme lediglich zur Übersetzungszeit und transformiert sie in JavaScript, wodurch sie unmittelbar in etablierten JavaScript-Laufzeitumgebungen ausgeführt werden können. Bierman, Abadi und Torgersen (2014) beschreiben TypeScript zudem als bewusst nicht vollständig statisch typischer ausgelegt, um gängige JavaScript-Idiome und die Typisierung existierender Bibliotheken praktisch zu unterstützen. [6]

Nach der Einführung in die Entstehungsgeschichte und die zentralen Entwurfsprinzipien von TypeScript soll im Folgenden die Programmiersprache selbst vorgestellt werden. Ziel ist es, einen Überblick über die grundlegenden Sprachkonstrukte zu geben, welche für die Vorlesungsunterlagen relevant sind.

Die Darstellung orientiert sich dabei am Notebook [Introduction.ipynb](#), das als Einstieg der Vorlesung *Theoretische Informatik* dient. Anhand dieses Notebooks werden zentrale Konzepte wie Variablendeklarationen, Typannotationen, Funktionen, Kontrollstrukturen, Arrays, Objekte sowie grundlegende Aspekte des Typsystems eingeführt und anhand von

Codebeispielen illustriert.

3.1 Primitive Datentypen und Arithmetik

In TypeScript, wie auch in JavaScript, werden alle numerischen Werte standardmäßig als Gleitkommazahlen gemäß dem [IEEE-754-Standard](#) repräsentiert. Es existiert kein separater Datentyp für Ganzzahlen (Integers). Dies gilt selbst dann, wenn die literale Darstellung eine Ganzzahl vermuten lässt.

```
1  typeof 666    // Ausgabe: number
2  typeof 666.0  // Ausgabe: number
```

Listing 7: Typüberprüfung numerischer Literale

Diese Designentscheidung hat direkte Auswirkungen auf die Präzision arithmetischer Operationen. Die Genauigkeit für Ganzzahlen ist durch die Konstante `Number.MAX_SAFE_INTEGER` begrenzt, welche den Wert $2^{53} - 1$ (ca. 9 Milliarden) repräsentiert. Jenseits dieser Grenze ist die exakte Darstellung von Ganzzahlen nicht mehr gewährleistet, was zu Präzisionsverlusten führt.

```
1  Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2
2  // Ausgabe: true
```

Listing 8: Demonstration des Präzisionsverlusts bei Number

Um die Limitierungen des `Number`-Typs zu umgehen, gibt es den primitive Datentyp `BigInt`. Er ermöglicht die Darstellung von Ganzzahlen beliebiger Größe, beschränkt lediglich durch den verfügbaren Arbeitsspeicher. Syntaktisch werden `BigInt`-Literals durch das Suffix `n` oder über den Konstruktor `BigInt()` erzeugt.

Ein wichtiger Aspekt der Typsicherheit in TypeScript ist die strikte Trennung dieser beiden numerischen Typen. Es ist nicht möglich, `BigInt` und `Number` in arithmetischen Operationen ohne explizite Konvertierung zu mischen. Zudem unterscheidet sich das Verhalten bei der Division: Während `Number` stets Gleitkommaergebnisse liefert, führt die Division von `BigInt`-Werten zu einer Trunkierung des Dezimalanteils (Ganzzahldivision).

```
1  const result: bigint = 7n ** 125n;
2  // Berechnung extrem großer Potenzen
3
```

```
4 console.log(5n / 2n);  
5 // Ausgabe: 2n (Nachkommastellen werden abgeschnitten)
```

Listing 9: Verwendung von BigInt

3.2 Ausgabe und Zeichenkettenlitterale

Für die Ausgabe von Werten auf der Konsole stellt TypeScript die Funktion `console.log` bereit. Diese Funktion ist polymorph und akzeptiert Argumente beliebigen Typs, um sie in einer textuellen Repräsentation auszugeben. Ein elementarer Datentyp für textuelle Daten ist der `String`. In TypeScript können Zeichenkettenlitterale äquivalent durch einfache Anführungszeichen (') oder doppelte Anführungszeichen (") definiert werden. Es gibt semantisch keinen Unterschied zwischen diesen beiden Notationen, was Entwicklern Flexibilität bei der Einhaltung von Coding-Style-Guidelines gewährt.

In der interaktiven Umgebung eines Jupyter Notebooks wird zudem das Ergebnis des letzten Ausdrucks einer Zelle automatisch ausgegeben, ohne dass ein expliziter Aufruf von `console.log` notwendig ist.

```
1 // Explizite Ausgabe über die Konsole  
2 console.log('Hello, World!');  
3  
4 // Implizite Ausgabe des letzten Ausdrucks im Notebook  
5 'Hello, World!'  
6  
7 // Alternative Syntax mit doppelten Anführungszeichen  
8 Hello, World!
```

Listing 10: Ausgabe und Definition von Strings

Template String

Zusätzlich zu den klassischen Zeichenkettenlitteralen unterstützt TypeScript sogenannte *Template Strings*. Diese werden durch Backticks (‘) umschlossen und bieten erweiterte Funktionalitäten gegenüber herkömmlichen Strings. Ein zentrales Merkmal ist die String-Interpolation (*String Interpolation*), die es ermöglicht, Ausdrücke direkt in eine Zeichenkette einzubetten. Hierbei wird die Syntax `${...}` verwendet, wobei der Inhalt der geschweiften Klammern ausgewertet und das Ergebnis in den String konvertiert wird.

Dies erhöht die Lesbarkeit des Codes erheblich, da komplexe Verkettungsoperationen vermieden werden. Zudem unterstützen Template Strings mehrzeilige Zeichenketten ohne spezielle Escape-Sequenzen.

```
1  const name = Alice ;
2  const age = 25;
3
4  // Direkte Einbettung von Variablen
5  console.log(`User ${name} is ${age} years old.`);
6  // Ausgabe: User Alice is 25 years old.
7
8  // Auswertung von Ausdrücken innerhalb des Strings
9  console.log(`Next year, she will be ${age + 1}.`);
10 // Ausgabe: Next year, she will be 26.
```

Listing 11: Verwendung von Template Strings

3.3 Arithmetische Operationen und Variablenzuweisung

Ein wesentlicher Unterschied zu vielen anderen Programmiersprachen betrifft die Division. In TypeScript liefert der Standard-Divisionsoperator (/) stets eine Gleitkommazahl als Ergebnis, auch wenn beide Operanden Ganzzahlen sind. Um eine Ganzzahldivision (*Integer Division* oder Euklidische Division) durchzuführen, muss das Ergebnis explizit abgerundet werden. Hierfür steht die Funktion `Math.floor()` zur Verfügung, welche die größte Ganzzahl zurückgibt, die kleiner oder gleich dem übergebenen Argument ist. Der Modulo-Operator % berechnet den Rest dieser Division.

```
1  // Standard-Division (Gleitkomma)
2  7 / 3; // Ergibt 2.333...
3
4  // Ganzzahldivision
5  Math.floor(7 / 3); // Ergibt 2
6
7  // Rest der Division (Modulo)
8  7 % 3; // Ergibt 1
```

Listing 12: Ganzzahldivision und Modulo

Zur Speicherung von Werten unterscheidet TypeScript zwischen Konstanten und Variablen. Das Schlüsselwort `const` deklariert eine Konstante, deren Wert nach der Initialisierung nicht mehr verändert werden kann (Reassigning is forbidden). Dies fördert die Unveränderlichkeit (*Immutability*) von Daten, wo immer es möglich ist. Soll sich ein Wert im Laufe des Programms ändern, muss die Variable mit dem Schlüsselwort `let` deklariert werden.

```
1  // Deklaration von Konstanten
2  const q = Math.floor(7 / 3);
3  const r = 7 % 3;
4  q = 5; // Error: Assignment to constant variable.
5
6  // Deklaration veränderlicher Variablen
7  let count = 1;
8  count = 2; // Erlaubt, da 'count' mit 'let' deklariert wurde
```

Listing 13: Konstanten vs. Variablen

3.4 Arrays

Arrays sind eine fundamentale Datenstruktur in TypeScript, die eine geordnete Sammlung von Elementen repräsentiert. Sie entsprechen konzeptionell den Listen in Python, unterscheiden sich jedoch in Syntax und bestimmten Verhaltensweisen. Ein Array wird in TypeScript durch eckige Klammern `[` und `]` definiert. Das leere Array, also ein Array ohne Elemente, wird wie folgt notiert:

```
1  []
```

Listing 14: Definition eines leeren Arrays

Ein Array mit konkreten Elementen wird durch Komma-separierte Werte innerhalb der eckigen Klammern erzeugt. Leerzeichen nach den Kommas sind optional, werden jedoch zur besseren Lesbarkeit empfohlen.

```
1  [1, 2, 3]
```

Listing 15: Array mit numerischen Elementen

Zugriff auf Array-Elemente: Der Index-Operator

Der *Index-Operator* `[.]` ist ein Postfix-Operator, der verwendet wird, um auf das Element an einer bestimmten Position (Index) zuzugreifen. TypeScript verwendet, wie die meisten modernen Programmiersprachen, eine nullbasierte Indizierung (*zero-based indexing*), d. h., das erste Element eines Arrays hat den Index 0.

```
1  const L = [1, 2, 3];  
2  L[0]; // Gibt das erste Element zurück: 1
```

Listing 16: Zugriff auf Array-Elemente

Der Index-Operator kann auch auf der linken Seite einer Zuweisung verwendet werden, um ein bestehendes Element zu modifizieren:

```
1  L[0] = 7;  
2  L; // Ergebnis: [7, 2, 3]
```

Listing 17: Modifikation eines Array-Elements

Verkettung von Arrays

Arrays können mittels der Methode `concat` verkettet werden. Diese Methode erstellt ein neues Array, das alle Elemente des aufrufenden Arrays gefolgt von allen Elementen des als Argument übergebenen Arrays enthält.

```
1  [1, 2, 3].concat([4, 5, 6]); // Ergebnis: [1, 2, 3, 4, 5, 6]
```

Listing 18: Verkettung mit `concat`

Alternativ kann der *Spread-Operator* `...` verwendet werden, um Arrays zu entpacken und zu kombinieren. Diese Syntax ist kompakter und wird in TypeScript häufig bevorzugt.

```
1  [...[1, 2, 3], ...[4, 5, 6]]; // Ergebnis: [1, 2, 3, 4, 5, 6]
```

Listing 19: Verkettung mit dem Spread-Operator

Die Eigenschaft `length` gibt die Anzahl der Elemente in einem Array zurück. Formal ist die Länge eines Arrays L definiert als $|L|$, die Kardinalität der Menge der enthaltenen Elemente.

```
1  [4, 5, 4].length; // Ergebnis: 3
```

Listing 20: Bestimmung der Array-Länge

Die Methode `filter`

Die Methode `filter` ist eine Higher-Order-Funktion, die ein neues Array erzeugt, welches nur jene Elemente des ursprünglichen Arrays enthält, für die eine übergebene Prädikatsfunktion `true` zurückgibt. Diese Methode implementiert das funktionale Konzept der *Filterung* und ist ein zentrales Werkzeug für deklarative Array-Transformationen. Die formale TypeScript Signatur sieht wie folgt aus:

```
1  filter(  
2  callbackfn: (value: number, index: number, array: number[]) =>  
    unknown,  
3  thisArg?: any  
4  ): number[]
```

Listing 21: Signatur `filter`

Die Callback-Funktion erhält automatisch drei Argumente:

1. `value`: Das aktuell verarbeitete Element
2. `index`: Der nullbasierte Index des Elements im Array
3. `array`: Das ursprüngliche Array, auf dem `filter` aufgerufen wurde

Im folgenden Beispiel werden nur gerade Zahlen aus dem Array extrahiert. Die Bedingung `n % 2 === 0` prüft, ob der Rest der Division durch 2 gleich 0 ist.

```
1  let K = [1, 2, 3, 4];  
2  const result = K.filter((n, i, k) => {  
3    console.log(n); // Aktuelles Element  
4    console.log(i); // Index  
5    console.log(k); // Ursprüngliches Array  
6    return n % 2 === 0;  
7  });  
8  result; // Ergebnis: [2, 4]
```

Listing 22: Filtern gerader Zahlen

Die Ausführung lässt sich wie folgt visualisieren:

Element (n)	Operation (n % 2)	Test (=== 0)	Ergebnis
1	1 % 2 = 1	1 === 0 → false	Drop
2	2 % 2 = 0	0 === 0 → true	Keep
3	3 % 2 = 1	1 === 0 → false	Drop
4	4 % 2 = 0	0 === 0 → true	Keep

Erzeugung von Arrays mit `Array.from`

Neben der literalen Schreibweise bietet TypeScript die statische Methode `Array.from`, um Arrays dynamisch zu erzeugen. Diese Methode ist besonders nützlich, um Arrays einer bestimmten Länge zu initialisieren und gleichzeitig mit Werten zu befüllen. Sie akzeptiert ein *Array-ähnliches* Objekt (ein Objekt mit einer `length`-Eigenschaft) und eine optionale Mapping-Funktion. Die formale TypeScript Signatur sieht wie folgt aus:

```

1  static from<T, U>(  

2    arrayLike: ArrayLike<T>,  

3    mapfn: (element: T, index: number) => U  

4  ): U[]

```

Listing 23: Signatur `from`

Im folgenden Beispiel wird ein Array erzeugt, das die Zahlenfolge von 1 bis 10 enthält:

```

1  Array.from({length: 10}, (_, i) => i + 1);  

2  // Ergebnis: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```

Listing 24: Erzeugung einer Zahlenfolge

Die Argumente werden dabei wie folgt interpretiert:

1. `{length: 10}`: Ein Objekt, das ein Array der Länge 10 simuliert. Da keine Elemente vorhanden sind, werden diese als `undefined` betrachtet.
2. `(_, i) => i + 1`: Die Mapping-Funktion. Der Unterstrich `_` ist eine Konvention für ein ignoriertes Argument (hier das `undefined`-Element). `i` ist der aktuelle Index (0 bis 9), der transformiert wird ($0 \rightarrow 1, \dots$).

Die Methode `map`

Die Methode `map` transformiert jedes Element eines Arrays mithilfe einer Callback-Funktion und gibt ein neues Array zurück, das die Ergebnisse dieser Transformation enthält. Die Länge des Arrays bleibt dabei unverändert.

```

1  const K = [1, 2, 3];
2  K.map(n => n * n);
3  // Ergebnis: [1, 4, 9]

```

Listing 25: Quadrieren von Zahlen mit map

Die Methode reduce

Die Methode `reduce` wird verwendet, um ein Array auf einen einzelnen Wert zu reduzieren (z. B. die Summe aller Elemente). Sie iteriert über das Array und führt einen Akkumulator mit, der den Zwischenstand der Berechnung speichert. Die formale TypeScript Signatur sieht wie folgt aus:

```

1  reduce(
2  callbackfn: (accumulator: number, currentValue: number) =>
    number,
3  initialValue: number
4  ): number

```

Listing 26: Signatur from

Die Callback-Funktion nimmt zwei Hauptargumente entgegen:

- `accumulator` (a): Der akkumulierte Wert (Startwert oder Ergebnis der vorherigen Iteration).
- `currentValue` (b): Das aktuelle Element des Arrays.

Das folgende Beispiel berechnet die Summe der Zahlen 1, 2 und 3:

```

1  [1, 2, 3].reduce((a, b) => a + b, 0);
2  // Ergebnis: 6

```

Listing 27: Summierung mit reduce

Der Ablauf der Berechnung (*Trace*) stellt sich wie folgt dar:

Schritt	a (Akkumulator)	b (Wert)	Operation (a + b)	Neu a
Start	0	—	—	0
1	0	1	0 + 1	1
2	1	2	1 + 2	3
3	3	3	3 + 3	6

Array Destructuring

Ein leistungsfähiges Feature von TypeScript ist das *Array Destructuring*. Es ermöglicht das „Entpacken“ (*unpacking*) von Werten aus einem Array direkt in separate Variablen, anstatt mühsam über den Index (wie `arr[0]`, `arr[1]`) darauf zuzugreifen. Dies führt zu kompakterem und lesbarerem Code, insbesondere wenn Funktionen mehrere Werte in Form eines Arrays zurückgeben.

```

1  let a, b, rest;
2  [a, b] = [10, 20];
3  console.log(a); // Ausgabe: 10
4  console.log(b); // Ausgabe: 20

```

Listing 28: Einfaches Array Destructuring

Destructuring kann auch mit dem *Rest-Operator* (`...`) kombiniert werden, um die verbleibenden Elemente eines Arrays in einer neuen Array-Variable zu sammeln. Dies ist besonders nützlich, um den „Kopf“ (Head) und den „Rest“ (Tail) einer Liste zu trennen, ein gängiges Muster in der funktionalen Programmierung.

```

1  [a, b, ...rest] = [10, 20, 30, 40, 50];
2  console.log(rest);
3  // Ausgabe: [30, 40, 50]

```

Listing 29: Destructuring mit Rest-Operator

3.5 Tupel

Ein *Tupel* ist in TypeScript ein spezieller Array-Typ, der eine feste Anzahl von Elementen mit vorab definierten Datentypen besitzt. Im Gegensatz zu herkömmlichen Arrays, die meist homogen typisiert sind (z. B. `string[]`), erlauben Tupel die Speicherung einer heterogenen Sammlung von Werten in einer strikten Reihenfolge. Dieses Konzept eignet

sich besonders, um zusammengehörige Daten zu gruppieren, ohne dafür eigens ein komplexes Objekt definieren zu müssen, beispielsweise für Koordinatenpaare `[x, y]` oder Rückgabewerte einer Funktion.

Im folgenden Beispiel wird ein Tupel-Typ `userProfile` definiert, der exakt drei Elemente in der Reihenfolge `number`, `string` und `boolean` erwartet:

```
1 let userProfile: [number, string, boolean]; // Typ-Definition
2 userProfile = [101, Alice, true];
3 // Zuweisung: Reihenfolge und Typen müssen exakt übereinstimmen
```

Listing 30: Definition und Initialisierung eines Tupels

Der Zugriff auf die Elemente erfolgt analog zu Arrays über den Index-Operator:

```
1 console.log( Name: , userProfile[1]); // Ausgabe: Name: Alice
```

Listing 31: Zugriff auf Tupel-Elemente

3.6 Objekte

Während Arrays und Tupel Werte in einer geordneten Liste speichern, dienen *Objekte* der Speicherung von Schlüssel-Wert-Paaren (*Key-Value Pairs*). Ein Objekt ist eine Sammlung von Eigenschaften (*Properties*), wobei jede Eigenschaft einen Namen und einen zugehörigen Wert besitzt. In TypeScript werden Objekte mit geschweiften Klammern `{}` definiert.

Das nachstehende Beispiel zeigt ein `student`-Objekt mit grundlegenden Eigenschaften:

```
1 const student = {
2   firstName: Marvin ,
3   matriculationNumber: 123456,
4   isEnrolled: true
5 };
```

Listing 32: Objektdefinition

Zugriff auf Eigenschaften

Es existieren zwei gängige Notationen für den Zugriff auf Objekteigenschaften:

- **Punkt-Notation** (`obj.key`): Die Standard-Methode für den Zugriff, wenn der Schlüsselname bekannt und ein gültiger Bezeichner ist.

- **Klammer-Notation** (`obj["key"]`): Notwendig, wenn der Schlüssel in einer Variable gespeichert ist oder Sonderzeichen enthält.

```

1 console.log(student.firstName); // Punkt-Notation
2 console.log(student[ matriculationNumber ]); // Klammer-
  Notation

```

Listing 33: Zugriffsarten auf Objekteigenschaften

Objekte sind veränderlich (*mutable*), sodass Werte bestehender Eigenschaften nachträglich modifiziert werden können:

```

1 student.isEnrolled = false;

```

Listing 34: Modifikation von Eigenschaften

Verschachtelte Objekte

Objekte können beliebig verschachtelt werden, um komplexe Datenstrukturen abzubilden. Das Beispiel `university` demonstriert eine Hierarchie, in der die Eigenschaft `location` selbst ein Objekt ist und `departments` ein Array von Strings darstellt.

```

1 const university = {
2   name: HTWG Konstanz ,
3   location: {
4     street: Alfred-Wachtel-Str. 8 ,
5     city: Konstanz ,
6     zipCode: 78462
7   },
8   departments: [ Computer Science , Architecture ,
9     Mechanical Engineering ],
10  isPublic: true
11 };

```

Listing 35: Verschachtelte Objekte

Der Zugriff auf tiefere Ebenen erfolgt durch Verkettung des Punkt-Operators bzw. Kombination mit dem Array-Index:

```

1 // Zugriff auf verschachteltes Objekt
2 university.location.city; // Ausgabe: Konstanz

```

```

3
4 // Zugriff auf Array innerhalb eines Objekts
5 university.departments[1]; // Ausgabe: Architecture

```

Listing 36: Zugriff auf verschachtelte Daten

3.7 Boolesche Werte und Operatoren

Die Repräsentation von Wahrheitswerten in TypeScript erfolgt über den primitiven Datentyp `boolean`, der die Werte `true` und `false` annehmen kann. Diese Werte sind zentral für die Steuerung des Programmflusses und logische Entscheidungen.

Logische Operatoren

TypeScript unterstützt die klassischen logischen Verknüpfungen Konjunktion (AND), Disjunktion (OR) und Negation (NOT) mittels der folgenden Operatoren:

```

1 true && false; // false
2 true || false; // true
3 !true; // false

```

Listing 37: Logische Verknüpfungen

Vergleichsoperatoren

Zur Erzeugung von Wahrheitswerten werden häufig numerische Vergleiche herangezogen. TypeScript bietet hierfür eine Reihe von Operatoren:

- `===`: Strikte Gleichheit (prüft Wert und Typ)
- `!==`: Strikte Ungleichheit
- `<`, `>`, `<=`, `>=`: Klassische Größenvergleiche

Ein wichtiger Unterschied zu Python besteht in der Verkettung von Vergleichen. Ein Ausdruck wie `1 < 2 < 3` wird in TypeScript nicht mathematisch ausgewertet. Stattdessen würde `1 < 2` zu `true` evaluiert, und anschließend versucht, `true < 3` zu berechnen (was durch Typ-Koerzition zu `1 < 3 → true` führen kann, aber oft nicht das Gewünschte ist). Korrekte mathematische Bereichsprüfungen müssen explizit mit `&&` verknüpft werden

Quantoren auf Arrays

TypeScript bietet Methoden auf Arrays, die den logischen Quantoren aus der Prädikatenlogik entsprechen:

1. **Allquantor** ($\forall$): Die Methode `every` prüft, ob *alle* Elemente eines Arrays eine Bedingung erfüllen.

```
1  const L = [2, 4, 0, 7];  
2  // Sind alle Zahlen gerade? -> false, da 7 ungerade ist  
3  L.every(x => x % 2 === 0);
```

2. **Existenzquantor** ($\exists$): Die Methode `some` prüft, ob *mindestens ein* Element die Bedingung erfüllt.

```
1  // Ist mindestens eine Zahl ungerade? -> true  
2  L.some(x => x % 2 !== 0);
```

Diese Methoden ermöglichen eine deklarative und mathematisch fundierte Ausdrucksweise für Mengenoperationen direkt im Code.

3.8 Kontrollstrukturen

Kontrollstrukturen dienen dazu, den Ablauf eines Programms zu steuern. TypeScript bietet hierfür klassische Konstrukte, die syntaktisch stark an C, Java oder C++ angelehnt sind. Die grundlegende bedingte Ausführung erfolgt mittels `if`, `else if` und `else`. Im Gegensatz zu Python müssen die Bedingungen in runde Klammern `()` gesetzt und die Anweisungsblöcke in geschweifte Klammern `{}` eingeschlossen werden.

```
1  let x = 17;  
2  if (x < 0) {  
3      console.log('The number is negative!');  
4  } else if (x === 0) {  
5      console.log('The number is zero.');
```

```
6  } else {  
7      console.log('It's more than one. ');  
8  }
```

Listing 38: If-Else Verzweigung

Alternativ bietet die `switch`-Anweisung eine strukturierte Möglichkeit, mehrere Bedingungen zu prüfen. Ein interessantes Muster in TypeScript ist das `switch(true)`-Konstrukt, das es erlaubt, komplexe Ausdrücke in den `case`-Zweigen auszuwerten:

```

1  switch (true) {
2      case (x < 0):
3          console.log('The number is negative!');
4          break;
5      case (x === 0):
6          console.log('The number is zero. ');
7          break;
8      default:
9          console.log(' It's more than one. ');
10 }

```

Listing 39: Switch-Statement als Alternative

Zur Iteration stellt TypeScript verschiedene Schleifenkonstrukte bereit:

- Die `for...of`-Schleife eignet sich ideal, um über die Elemente eines Arrays oder anderer iterierbarer Objekte zu laufen. Sie ist kompakter und weniger fehleranfällig als die klassische indexbasierte `for`-Schleife.
- Die `while`-Schleife führt einen Codeblock solange aus, wie eine definierte Laufbedingung erfüllt ist.

Anwendungsbeispiel: Sieb des Eratosthenes

Ein klassischer Algorithmus zur Bestimmung aller Primzahlen bis zu einer Grenze n ist das Sieb des Eratosthenes. Die Implementierung in TypeScript demonstriert das Zusammenspiel von Arrays, Schleifen und bedingten Anweisungen:

1. Initialisierung eines Arrays `primes`, wobei `primes[i] = i` gilt.
2. Iteration über alle Zahlen i und j , um Vielfache zu eliminieren (d. h. auf 0 zu setzen).
3. Filterung der verbleibenden Zahlen $\neq 0$.

Eine optimierte Version nutzt zudem `Math.sqrt(n)` als Obergrenze für die äußere Schleife und beginnt die innere Schleife bei $j = i$, um redundante Prüfungen zu vermeiden.

```

1  let n = 10000;
2  // Initialisiere Array mit Werten 0 bis n
3  let primes = Array.from({length: n + 1}, (_, i) => i);
4  let limit = Math.ceil(Math.sqrt(n));
5  for (let i = 2; i <= limit; i++) {

```

```

6      // Wenn i bereits markiert (0) ist, überspringen
7      if (primes[i] === 0) continue;
8      let j = i;
9      // Streiche alle Vielfachen von i (beginnend bei i*i)
10     while (i * j <= n) {
11         primes[i * j] = 0;
12         j++;
13     }
14 }
15 // Filtere alle Nicht-Null-Werte (Primzahlen)
16 let result = primes.filter(p => p !== 0 && p >= 2);

```

Listing 40: Optimiertes Sieb des Eratosthenes

3.9 Numerische Funktionen

TypeScript stellt, über das zugrundeliegende JavaScript, eine umfassende Sammlung mathematischer Standardfunktionen und Konstanten bereit, die im globalen `Math`-Objekt gekapselt sind. Dieses Objekt fungiert als Namensraum für statische Methoden und Eigenschaften. Zu den elementaren Funktionen gehören:

- **Wurzelberechnung:** `Math.sqrt(x)` berechnet die Quadratwurzel $\sqrt{x}$.
- **Betragsfunktion:** `Math.abs(x)` liefert den absoluten Wert $|x|$.
- **Potenzierung:** `Math.pow(base, exponent)` oder der Exponentialoperator `**`.

Rundungsfunktionen

Für die Umwandlung von Gleitkommazahlen in Ganzzahlen stehen drei spezifische Methoden zur Verfügung:

1. `Math.floor(x)`: Die *Gaußklammer unten* $\lfloor x \rfloor$. Sie rundet stets auf die nächstkleinere ganze Zahl ab.

$$\text{floor}(x) = \max\{n \in \mathbb{Z} \mid n \leq x\}$$

2. `Math.ceil(x)`: Die *Gaußklammer oben* $\lceil x \rceil$. Sie rundet stets auf die nächstgrößere ganze Zahl auf.

$$\text{ceil}(x) = \min\{n \in \mathbb{Z} \mid n \geq x\}$$

3. `Math.round(x)`: Kaufmännisches Runden zur nächsten ganzen Zahl. Hierbei wird $n.5$ auf $n + 1$ gerundet.

Trigonometrische Funktionen

Das `Math`-Objekt umfasst auch die Standard-Trigonometriefunktionen wie `Math.sin`, `Math.cos` und `Math.tan` sowie deren Arkusfunktionen (`asin`, `acos`, `atan`). Es ist zu beachten, dass diese Funktionen Winkel im Bogenmaß (Radiant) erwarten bzw. zurückgeben.

Exponential- und Logarithmusfunktionen

Die Exponentialfunktion e^x wird über `Math.exp(x)` berechnet. Das Inverse, der natürliche Logarithmus $\ln(x)$, wird in TypeScript (wie in vielen C-basierten Sprachen) als `Math.log(x)` bezeichnet.

Konstanten

Wichtige mathematische Konstanten sind direkt zugänglich:

- `Math.PI` für die Kreiszahl $\pi \approx 3.14159$
- `Math.E` für die Eulersche Zahl $e \approx 2.718$

4 Methodische Vorgehensweise

In diesem Kapitel soll gezeigt werden, wie die Übersetzung der Notebooks erfolgt sind. Von dem zuerst grundlegenden Verständnis über die einzelnen Funktionen hin zu der unterstützenden Übersetzung mithilfe von KI bis hin zum vollständig funktionierenden Notebook

4.1 Github Projects

Hier möchte ich unser Projektstatus zeigen, eingehen, wie wir uns diese Tickets zugeteilt haben und wie wir mit der großen Menge an Notebooks fertig geworden sind.

4.2 Auswahl der Lehrmaterialien

Welche Kriterien es gab...

4.3 Der Übersetzungs-Workflow

Wann manuell, wann automatisiert

4.4 Einsatz von KI-gestützten Tools

Verwendete Modelle (GPT-4, Claude, Copilot etc.)

Prompt-Engineering-Strategien für Code-Übersetzung. (hier würde ich gerne unseren Prompt erwähnen und erläutern, vielleicht machen wir hierfür nochmal ein Unterkapitel)

Grenzen der KI (Halluzinationen bei seltenen TS-Libraries).

5 Case Studies

Im folgenden haben wir uns Fälle aus Notebooks herausgesucht, welche wir genauer aufzeigen möchten.

5.1 CNF

In diesem Teil wollen wir etwas genauer auf das Notebook 04-CNF eingehen und die Umsetzung des Notebooks und dessen Funktionen genauer unter die Lupe nehmen.

5.2 Graphviz

...

5.3 Wiederverwendung von Notebooks

...

5.4 Recursive Set

Erläuterung und Nutzen

6 Evaluierung und Ergebnisse

In diesem Kapitel werden wir auf den Erfolg der Übersetzung vergleichen.

6.1 Laufzeit

...

6.2 Lesbarkeit

...

6.3 Effizienz

-i z.B. 8 Damen Problem -i Code-Snipppets (so, dass sie relevant sind); allgemeine Suchalgorithmen)

```
1
2      // Ein Interface definieren
3      interface User {
4          id: number;
5          name: string;
6      }
7
8      // Eine Funktion mit Typisierung
9      function greet(user: User): string {
10         return `Hallo, ${user.name}!`;
11     }
12
13     const myUser: User = { id: 1, name: "Max" };
14     console.log(greet(myUser));
```

Listing 1: TypeScript: Beispiel mit Interface und Funktion

7 Fazit und Ausblick

Beispiel FOL-Parser viel zu großer Umfang

Literaturverzeichnis

- [1] *Algoithms Repository*. Website. URL: <https://github.com/karlstroetmann/Algorithms>. (Zugriff am 13. Oktober 2025).
- [2] *Logic Repository*. Website. URL: <https://github.com/karlstroetmann/Logic>. (Zugriff am 13. Oktober 2025).
- [3] *Teaching and Learning with Jupyter*. Website. URL: <https://jupyter4edu.github.io/jupyter-edu-book/>. (Zugriff am 19. Januar 2026).
- [4] *TypeScript*. Website. URL: <https://en.wikipedia.org/wiki/TypeScript>. (Zugriff am 25. Januar 2026).
- [5] *TypeScript Repository*. Website. URL: <https://github.com/Microsoft/TypeScript?tab=readme-ov-file>. (Zugriff am 25. Januar 2026).
- [6] *Understanding TypeScript*. Website. URL: https://link.springer.com/chapter/10.1007/978-3-662-44202-9_11. (Zugriff am 19. Januar 2026).